

ADULTERA — Project Plan

Smart Food Adulteration Detection & Evidence System

Beginner-friendly edition — every term and decision explained in plain language.

1. What Is This Project, In Simple Terms?

Food adulteration means mixing something cheap, fake, or harmful into food to increase profit or fake quality — for example, adding starch to milk to make it look thicker, or mixing colored powder into turmeric to make it look brighter than it really is.

Ordinary people often suspect something is wrong with a food product, but they usually don't know:

- How to actually test it at home
- What the test result even means once they see it
- How to keep proof (evidence) in case they want to complain
- Where or how to report it
- Whether anyone else has run into the same problem with the same product

ADULTERA is a web app that solves this by connecting all of these steps into one place. Instead of a person being stuck at "I think something's wrong but I don't know what to do," the app walks them through: Test → Understand the Result → Save Evidence → Spot Patterns With Others → Take Action.

Important: ADULTERA is not a food testing laboratory. It doesn't give scientifically certified proof. It gives people a starting point — a preliminary, guided way to check something and document it properly, so if it turns out to be a real problem, they already have organized evidence.

2. The Problem We're Solving (In Detail)

Right now, if someone suspects their milk or spices have been adulterated, here's what usually happens: they either do nothing, or they Google something vague and give up. The pieces they'd need — test instructions, complaint procedures — already exist somewhere on the internet, but they're scattered and disconnected. Nobody has connected them into a single guided experience.

So our project's real contribution isn't "inventing food testing" — it's **removing the friction between suspicion and action.**

3. What Success Looks Like

Here's how we'll know if the project is actually working, explained simply:

What We Want	How We'll Know We Got It
People can find a real test to run	At least 10 common foods (milk, turmeric, oil, etc.) have a documented test when the app launches
People can understand their result without guessing	The app gives an AI-based reading of the photo, with a confidence percentage (like "91% likely")
Evidence doesn't get lost or forgotten	Every case a user creates saves photos, batch number, and the purchase bill automatically
The app can spot a real pattern, not just noise	The "cluster" feature only triggers when enough separate reports line up — not from a single complaint
We never pretend to be a certified lab	Every single result screen shows a clear disclaimer saying this is preliminary, not official

4. The Six Main Features (Explained One at a Time)

This is the heart of the app. Each of these is a separate piece of functionality — think of them as six different "jobs" the app does for the user.

4.1 Adulteration Test Assistant — "Tell me how to test this"

What it does: The user picks a food (say, "Milk"). The app then shows them, step by step:

- What materials they need (e.g., a few drops of iodine)
- The exact steps to follow, in order
- What a normal result looks like (so they know what "fine" looks like)
- What a suspicious result looks like (so they know what to watch for)
- Safety warnings (e.g., "don't drink the test sample")
- A button to take a photo of their result at the end

Example, explained: For milk, the classic home test is to add a few drops of iodine solution. If the milk turns blue or black, that's a sign starch might have been added (pure milk won't react this way). The app would show all of this to the user clearly, then let them log their result.

Why it matters for beginners to understand: This feature needs to be built as a content system, not hardcoded pages. That means: instead of a programmer writing a separate webpage for "Milk Test," "Oil Test," "Turmeric Test," etc., we store all tests as structured data (like entries in a table or database) with the same fields (materials, steps, normal result, suspicious result). Then one generic page design can display any test just by loading the right data. This makes it easy to add a new test later without writing new code.

4.2 AI / Computer Vision Analysis — "Look at my photo and tell me what it might mean"

What it does: After the user takes a photo of their test result (like the milk turning blue), the app analyzes the image automatically and tells them something like:

```
Sample: Milk
Observed: Blue/black reaction
Possible finding: Starch detected
Confidence: 91%
⚠ This is a preliminary screening only. Lab confirmation is recommended.
```

In plain terms, what is "computer vision" here? It means using code that can look at the colors and patterns in a photograph and make a judgment — similar to how your phone can recognize a face in a picture, except here it's recognizing colors that suggest a chemical reaction happened.

Why this is the hardest part of the project (important to understand): Building an AI model that's trained to recognize adulteration reactions accurately would normally require thousands of labeled example photos — which we don't have. So instead of trying to build a full "trained machine learning model," a much more realistic and still legitimate approach for this project is:

- Use a rule-based color detector instead of a trained AI model. This means: the code checks the actual color values in the photo (for example, using a tool called OpenCV to detect "is this pixel range blue/black?") and applies a simple rule like "if more than X% of the test area is dark blue, flag as possible starch." This still counts as computer vision, is realistic to build in a student/team project timeframe, and can be explained honestly as a heuristic (a smart shortcut) rather than a fully trained AI model.

This distinction matters because it changes what kind of code the team writes and how the confidence score is calculated (from color-matching logic, not a trained neural network).

4.3 Evidence Vault — "Keep all my proof in one organized place"

What it does: Every time a user runs a test and suspects a problem, they can create a "case" — like a digital folder — that stores:

- Product name and brand
- Batch number (the code printed on the packaging that identifies which production run it came from)
- Purchase date, where it was bought, and location
- Which test was performed and what the result was
- Photos: of the product itself, and of the test result
- The purchase receipt/bill
- Optionally, a video

Example, explained: A case might look like this:

```
Case #ADL-2026-00482
Product: XYZ Milk
Batch: M240926
Test: Starch Test
Result: Suspected positive
Evidence attached: product photo ✓, batch number ✓, test photo ✓, bill ✓
Status: Awaiting verification
```

Why this matters: This turns a vague feeling ("I think something's off with this milk") into an organized record that could actually be useful — whether that's for the user's own records, for a formal complaint, or for spotting a pattern with other users' reports. This is also the central data table of the whole app — almost every other feature reads information from these cases.

4.4 Community Adulteration Heatmap — "Is this happening to other people too?"

What it does: Users can anonymously submit their suspected cases to a shared, public view. The app then shows this information as a map, grouped by area and food type — for example:

```
Pune
📦 Milk — 8 reports
📦 Spices — 4 reports
📦 Oil — 11 reports
📦 Rice — 1 report
```

Each report also moves through a status pipeline as it gets more serious: Reported → Evidence submitted → Lab verified → Resolved

An important rule to understand: A report being on the map does not mean the product is confirmed to be adulterated — it just means someone suspects it and logged a test. The app's design must make this distinction obvious everywhere (through labels, colors, and wording), so nobody mistakes "8 people reported this" for "this product is proven bad."

4.5 Adulteration Risk Score — "How worried should I be about this product?"

What it does: The app combines several pieces of evidence into a single score that gives a rough sense of risk. For example:

```
Product: XYZ Milk
Home test result: +40 points
Multiple reports: +20 points
Same batch reported by others: +15 points
Evidence attached: +10 points
Total: Risk = HIGH
```

In plain terms: Think of it like a credit score, but for "how suspicious is this specific product right now." The more independent, evidence-backed signals point the same way, the higher the score climbs.

Important design note: The exact point values (40, 20, 15, 10 above) should not be permanently hardcoded into the app's code. They should be stored as adjustable settings, because the team will likely need to tune these numbers after seeing how real test data behaves. This is clearly labeled to users as a software-generated estimate, not a scientific or legal conclusion.

4.6 Batch-Level Cluster Detection — Our standout feature

What it does: If multiple different users report the same product and the same batch number, the app automatically notices this and creates a special alert:

🔍 Possible Batch Cluster

Product: XYZ Turmeric

Batch: T24091

Reports: 5

Areas: 3

Tests performed: 4

Evidence files: 11

Status: Under review

Why this is the most important feature to highlight: Anyone can build a simple complaint form. What makes ADULTERA different is that it automatically looks for patterns — specifically, when the same batch of a product shows up in multiple unrelated reports, that's a much stronger signal than one person's complaint. This turns individual suspicions into a meaningful, trackable signal that something might be wrong with an entire production run, not just one bad bottle.

A build note for beginners: This only makes sense once a minimum number of separate reports exist (e.g., "trigger only after 3+ reports from 2+ different users"). This threshold needs to be decided and adjustable — it shouldn't fire on the very first report, or it becomes meaningless noise.

5. What the App Actually Looks Like (Screen by Screen)

Here's the basic navigation structure — think of this as the app's main menu:

Home	
🔍 Test Food	→ Walks the user through a test (Feature 4.1)
🔍 Analyze Result	→ Upload a photo for AI analysis (Feature 4.2)
🔍 My Cases	→ View/manage your own saved cases (Feature 4.3)
🔍 Risk Map	→ See the community heatmap (Feature 4.4)
🔍 Check Product	→ Search if a product/batch has existing reports
🔍 Test Library	→ Browse all available tests, without starting one

Each of these screens maps directly to one of the six features above, which makes it easier to divide work: one screen = one feature = one team member's responsibility, roughly.

6. Suggested Technology (With Plain Explanations)

This table lists what tools/technologies we suggest using, and — importantly — why, in plain terms for anyone not deeply familiar with web development.

Layer	Suggested Tool	What It's For, In Plain Words
Frontend (what the user sees/clicks)	React + Tailwind CSS	React is a popular tool for building interactive web pages out of reusable pieces ("components"). Tailwind is a styling tool that makes it fast to make things look clean without writing custom CSS from scratch.
Backend (the "brain" that handles requests)	Node.js + Express.js	This is the server — the part that receives requests from the app (like "save this case") and decides what to do with them, such as saving to the database.

Layer	Suggested Tool	What It's For, In Plain Words
Database (where all the data is stored)	MongoDB or Firebase	This is where cases, tests, and user data actually live, permanently. MongoDB stores data in a flexible, document-like format that fits well with how our "cases" are structured. Firebase is a faster option to get started with if the team wants less setup.
AI / Image Processing	Python + OpenCV (rule-based)	This is the part that looks at the uploaded test photos and analyzes their colors to make a judgment call. Runs separately from the main backend and gets called when needed.
Maps	Leaflet + OpenStreetMap	This displays the heatmap feature. Both are free and don't require signing up for a paid API key, which makes them good for a student/prototype project.
Login/Accounts	To be decided (see Section 7)	Needed so "My Cases" only shows a user's own cases, and to prevent spam reports on the community map.
Photo Storage	Firebase Storage, or local storage for v1	Every case includes photos, so we need somewhere reliable to store image files, separate from the main database (which is better suited for text/numbers).

7. Things We Need to Decide Before Building Starts

These are open questions the team should discuss and settle early, because they affect how the database and features are designed. Rushing past these usually causes rework later.

1. Should reporting be anonymous, or require an account?

Anonymous reporting is easier for users and encourages more people to report, but it makes it harder to prevent fake/spam reports. Requiring an account solves that but adds friction. This decision affects the database design, so it should be made before Phase 1 below.

2. How accurate can our AI analysis realistically be?

As explained in Section 4.2, we recommend a rule-based color detector rather than a fully trained AI model, since we don't have training data. This should be clearly communicated internally and to evaluators — it's a legitimate engineering choice, not a shortcut to hide.

3. What disclaimer language do we use, and where?

Since this project touches food safety, every screen that shows a result or risk score needs a consistent, clearly visible disclaimer (e.g., "Preliminary screening only — not a certified lab result"). It's much easier to build this once as a reusable piece (a shared component) and place it everywhere, rather than writing separate disclaimer text on each page.

4. What triggers a "cluster" alert?

As mentioned in Section 4.6, we need to agree on a minimum number of reports/areas before the batch-cluster alert fires. This number should be easy to adjust later, not buried in code.

5. How do we make the app look "alive" for a demo with no real users yet?

Features like the heatmap and cluster detection look empty and unconvincing with zero data. We should plan to create realistic sample/seed data specifically for demonstrations and presentations.

8. Suggested Build Plan (Phase by Phase)

This breaks the work into stages so the team can build incrementally instead of trying to do everything at once. Each phase builds on the previous one.

Phase 1 — Foundation (Week 1–2)

Goal: Get the basic skeleton of the app running.

- Set up the project repository (the shared code folder everyone works from)
- Set up the basic React frontend and Express backend, connected to each other
- Design the database structure for "Cases" and "Tests"
- Write the content for 5–10 tests (materials, steps, expected results) as sample data
- Build simple navigation between all six screens, even if most are empty placeholders for now

Phase 2 — Core Evidence Flow (Week 2–3)

Goal: Let a user actually run a test and save it as a case.

- Build the Test Assistant flow: pick a food → see the steps → take a photo
- Build the Evidence Vault: create a case, view it, edit it, see its status
- Add basic login/accounts so "My Cases" only shows that user's own cases

Phase 3 — Analysis & Scoring (Week 3–4)

Goal: Make the app "smart" — give users a reading and a risk score.

- Build the rule-based color-analysis tool (OpenCV) and connect it to photo uploads
- Calculate the Risk Score for each case based on the weighted formula (Section 4.5)
- Build one shared disclaimer component and use it on every results screen

Phase 4 — Community Features (Week 4–5)

Goal: Connect individual cases into a shared, useful bigger picture.

- Build the heatmap using Leaflet, pulling in anonymized case locations
- Build the batch-cluster detection logic and its alert card
- Build the "Check Product" search so users can look up an existing product/batch

Phase 5 — Polish & Demo Preparation (Week 5–6)

Goal: Make sure everything works together and looks convincing for a presentation.

- Add realistic sample data across multiple products, batches, and locations
- Test the full journey end-to-end: run a test → analyze it → save a case → see it show up on the map/cluster/risk score
- Prepare the pitch explanation (Section 9) and rehearse demoing the live app

9. How to Explain This Project When Asked

If someone (like a professor or evaluator) asks:

"Food adulteration detection already exists in some form. What is your project actually improving?"

Here's a clear, simple answer:

"ADULTERA creates a bridge between a consumer's initial suspicion and formal action. It guides users through standardized preliminary tests, analyzes visible test results, preserves evidence such as batch numbers and bills, identifies clusters of similar reports, and provides community-level analytics. It doesn't replace laboratories — it helps consumers collect better information and identify potential patterns."

The one-sentence version, if you only have a few seconds:

ADULTERA is a consumer-focused system that turns food-adulteration suspicion into structured testing, evidence, analysis, and actionable patterns.

10. Immediate Next Steps (What To Do Right Now)

- ☐ Decide: anonymous reporting vs. requiring user accounts (this blocks database design — see Section 7, point 1)
- ☐ Decide: confirm we're using the rule-based color-detection approach for the AI feature (see Section 7, point 2)
- ☐ Start now, no dependencies: write up the first 5–10 tests for the Test Library — this is a content-writing task anyone on the team can begin immediately
- ☐ Decide: set the initial threshold numbers for when a "batch cluster" alert should trigger
- ☐ Assign: who owns which of the 5 build phases above, so work can happen in parallel where possible